Introduction to C Programming

1 What is C?

- → C is a general-purpose programming language.
- → Developed by Dennis Ritchie in 1972.
- → Known for speed, efficiency, and portability.
- → Used in operating systems, embedded systems, and software development.

2 Features of C

3 Basic Program Structure

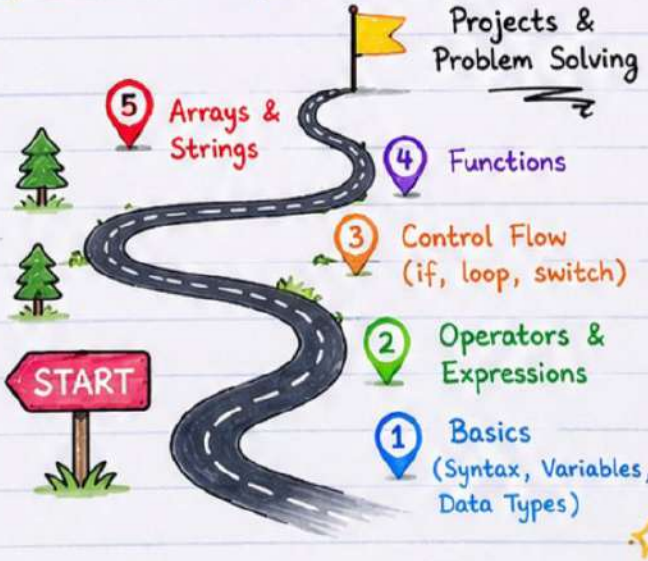
Every C program starts here!

```c
#include <stdio.h>
int main() {
    printf("Hello World");
    return 0;
}
```

4 Key Components

1 Header Files	→ Include library files in the program.
2 Main Function	→ Execution starts from main().
3 Statements	→ Instructions ended with ;
4 Functions	→ Reusable block of code.

5 C Learning Roadmap



C is the base,
Future is in your code!

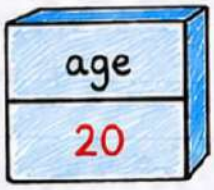
Variables & Data Types

Download Notes — coderstar.shop

1 Variable

- A variable stores data in memory.

```
int age = 20;
```



Think: Variable is a box that holds data!

2 Rules for Variables

- ✓ Cannot start with a number
- ✓ No spaces allowed
- ✓ Case-sensitive

Examples

1name	→	Wrong ✗
my name	→	Wrong ✗
Age	→	Valid ✓
age	→	Valid ✓
AGE	→	Valid ✓

3 Data Types

Data Type	Example	
int	10	→ Whole numbers 123
float	3.14	→ Decimal numbers 3.14
char	'A'	→ Single character 'A'
double	99.99	→ Large decimal numbers 99.99
void	No value	→ No value / emoty ---

Size (Bytes)

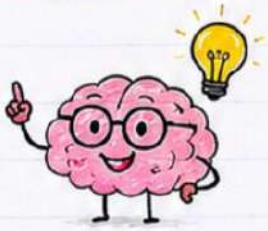
int	→	4
float	→	4
char	→	1
double	→	8
void	→	0

4 Constants

```
const float PI = 3.14;
```

→ Value cannot be changed.
→ Use 'const' keyword.

Remember: Variables can change, Constants cannot!



5 Keywords

Reserved words in C:

int	→	Used for integers
float	→	Used for decimal numbers
char	→	Used for characters
return	→	Return from function
if	→	Decision making

Keep Practicing, Keep Coding! ❤

Operators in C

Download Notes — coderstar.shop

Let's Code!

1 Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	7 - 2	5
*	Multiplication	4 * 6	24
/	Division	10 / 2	5
%	Modulus	10 % 3	1

% gives the remainder (after division)

2 Relational Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	✓
!=	Not equal to	5 != 3	✓
>	Greater than	7 > 2	✓
<	Less than	3 < 6	✓
>=	Greater than or equal to	4 >= 4	✓
<=	Less than or equal to	2 <= 5	✓

Result is either True (1) or False (0)

5 > 3

3 Logical Operators

Operator	Name	Example	Result
&&	AND	(5 > 3 && 2 < 4)	True
||	OR	(5 > 3 || 2 > 4)	True
!	NOT	!(5 > 3)	False

Truth Table

A	B	A && B	A || B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Logic is Fun!

4 Assignment Operators

Operator	Example	Same As
=	a = 5	a = 5
+=	a += 3	a = a + 3
-=	a -= 2	a = a - 2
*=	a *= 4	a = a * 4
/=	a /= 2	a = a / 2
%=	a %= 3	a = a % 3

Example

```
int a = 10;
a += 5;  // a = 15
a *= 2;  // a = 30
a -= 10; // a = 20
```

5 Increment & Decrement

Increment (++)

a++; → Increase by 1

a = 5 —a++→ a = 6

First use, then increase

Both change the value of variable by 1

Decrement (--)

a--; → Decrease by 1

a = 5 —a--→ a = 4

First use, then decrease

Example

```
int a = 5;
a++;  // a = 6
a--;  // a = 5
```

Operators make our code powerful
Practice More, Code Better!

Think
Learn
Code
Repeat

Conditional Statements in C

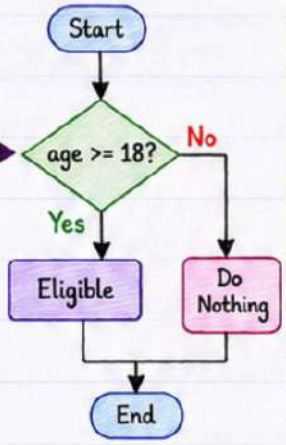
Download Notes — coderstar.shop

Make Decisions in Your Code!

Conditional statements help us make **decisions** in our program.

1 if Statement

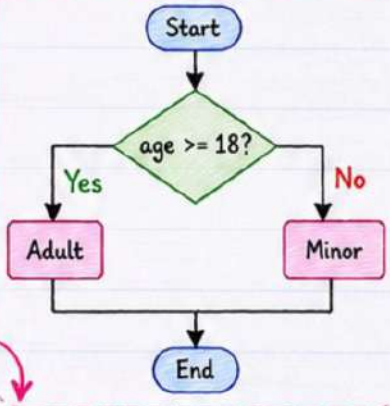
```
if (age >= 18)
{
    printf("Eligible");
}
```



If condition is TRUE, block runs.

2 if-else Statement

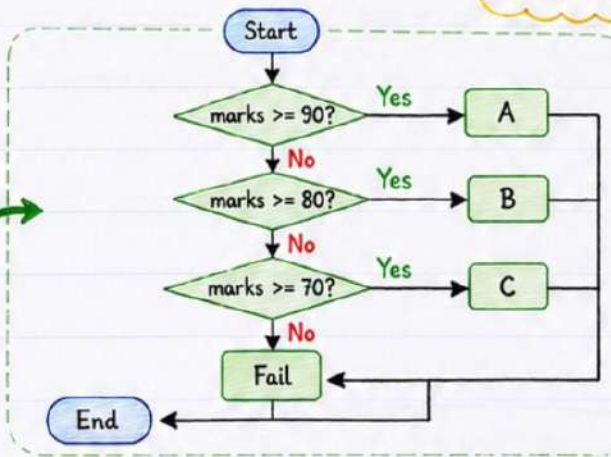
```
if (age >= 18)
{
    printf("Adult");
}
else
{
    printf("Minor");
}
```



If FALSE, else block runs.

3 else-if Ladder

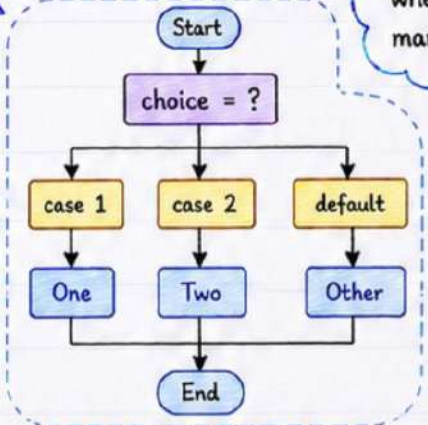
```
if (marks >= 90)
    printf("A");
else if (marks >= 80)
    printf("B");
else if (marks >= 70)
    printf("C");
else
    printf("Fail");
```



Good for multiple conditions!

4 switch Statement

```
switch (choice)
{
   case 1:
      printf("One");
      break;
   case 2:
      printf("Two");
      break;
   default:
      printf("Other");
}
```



Use switch when you have many choices!

Tips

- Conditions result in TRUE (1) or FALSE (0).
- Use proper indentation.
- break is important in switch.

Practice More, Code Better!

Think → Decide → Code → Succeed

Loops in C

Download Notes — coderstar.shop

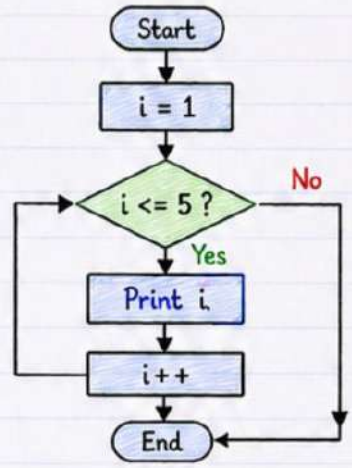
Loops help us repeat a block of code multiple times.

Loops Make Work Easy!

1 for Loop

```
for(int i = 1; i <= 5; i++)
{
    printf("%d", i);
}
```

Output
1 2 3 4 5

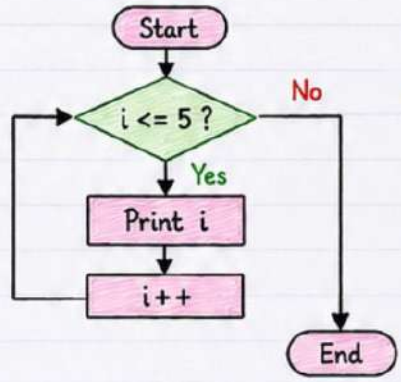


Syntax :

```
for (initialization;
    condition; increment)
{
    // code to repeat
}
```

2 while Loop

```
while(i <= 5)
{
    printf("%d", i);
    i++;
}
```

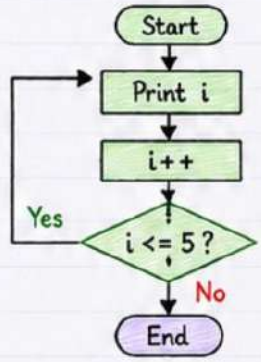


Check condition first, then execute the loop.

Output
1 2 3 4 5

3 do-while Loop

```
do
{
    printf("%d", i);
    i++;
}
while(i <= 5);
```



Execute once first, then check condition.

Output
1 2 3 4 5

4 Loop Control

break;

- Terminates the loop.

```
for(i = 1; i <= 5; i++)
{
   if(i == 3)
      break;   // exit loop
   printf("%d", i);
}
```

Output
1 2

continue;

- Skips the rest of the loop and continues with next iteration.

```
for(i = 1; i <= 5; i++)
{
   if(i == 3)
      continue;   // skip 3
   printf("%d", i);
}
```

Output
1 2 4 5

Remember!

- ☑ Use loops to save time
- ☑ Avoid writing same code again
- ☑ Practice makes perfect!

Write Once, Run Many Times!

Practice Loops Today, Code Like a Pro Tomorrow! ☺

Arrays & Strings

Download Notes — coderstar.shop

1 Array

- Stores multiple values of the same type.

```
int num[5] = { 1, 2, 3, 4, 5 };
```

	num[0]	num[1]	num[2]	num[3]	num[4]
num →	1	2	3	4	5
Index →	0	1	2	3	4

Think!
Array index always starts from 0.

2 Access Elements

- Use index to access array elements.
- num[0] → 1
- num[1] → 2
- num[2] → 3
- ...
- num[4] → 5

Example

```
printf("%d", num[1]);
```

Output: 2

3 String

- Character array ending with '\0' (null character).

```
char name[] = "CoderStar";
```

name →	C	o	d	e	r	S	t	a	r	\0
	0	1	2	3	4	5	6	7	8	9

Null Terminator

Note
"\0" marks the end of the string.

4 String Functions

Function	Use	Example	Demo
strlen(s)	Length of string	strlen("Hello")	→ 5
strcpy(dest, src)	Copy string	strcpy(a, "C");	a = "C"
strcat(dest, src)	Concatenate strings	strcat(a, "Star");	a = "CStar"
strcmp(s1, s2)	Compare strings	strcmp("A", "B");	→ Negative

Include <string.h> header file to use these functions.

5 Benefits

- ☑ Easy data management
  Store and access many values easily.
- ☑ Reduced code repetition
  Use loops and functions with arrays & strings.

Arrays & Strings make our programs powerful and efficient!

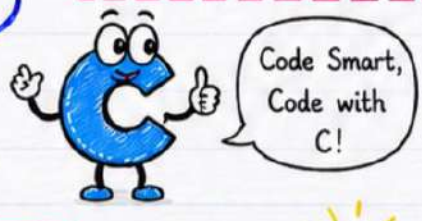
Practice More, Code Better!

☆ Master Arrays & Strings ☆ Build Better Programs! ☆

Functions & Pointers

Download Notes — coderstar.shop

Functions make our code modular, Pointers make it powerful!



1 Function

- Reusable block of code.

```c
#include <stdio.h>
void greet()
{
    printf("Hello");
}
```

Why Functions?
- ☑ Reusability
- ☑ Better Readability
- ☑ Easy Maintenance

2 Function Types

1. No Arguments, No Return

```c
void hello()
{
    printf("Hi");
}
Call → hello();
```

2. Arguments, No Return

```c
void printNum(int n)
{
    printf("%d", n);
}
Call → printNum(10);
```

3. No Arguments, Return

```c
int getValue()
{
    return 100;
}
Call → int x = getValue();
```

4. Arguments, Return

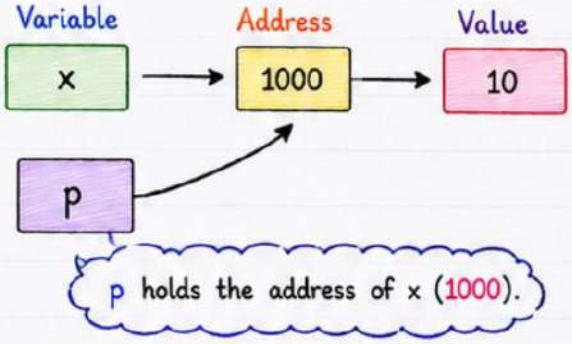
```c
int add(int a, int b)
{
    return a + b;
}
Call → int sum = add(5, 3);
```

3 Pointer

- Stores memory address.

```c
int x = 10;
int *p = &x;
```

Here, p is a pointer to x.



4 Pointer Operators

Operator	Name	Use	Example
&	Address Of	Returns address of a variable	int *p = &x;
*	Dereference (Indirection)	Accesses the value at the address	printf("%d", *p);

5 Example

```c
#include <stdio.h>
int main()
{
    int x = 10;
    int *p = &x;
    printf("Value of x = %d\n", x);
    printf("Address of x = %u\n", &x);
    printf("Value using pointer = %d\n", *p);
    return 0;
}
```

Output

```
Value of x = 10
Address of x = 1000
Value using pointer = 10
```

Memory Magic!

6 Advantages

☑ **Dynamic Memory**
Pointers help in dynamic memory allocation (malloc, free).

☑ **Efficient Programming**
Functions reduce code size and improve performance.

Functions give structure, Pointers give power!

Keep Practicing, Keep Improving!

Structures, File Handling in C

Download Notes — coderstar.shop

Organize Data, Manage Files!

1 STRUCTURES

→ Structure groups different data types under one name.

```
struct Student
{
    int id;
    char name[20];
};
```

Definition of Structure

Example

```
struct Student s1;
s1.id = 101;
strcpy(s1.name, "CoderStar");
```

Memory Representation

s1.id	s1.name								
101	C	o	d	e	r	S	t	a	r	\0
Index	0	1	2	3	4	5	6	7	8	9	10

Accessing Members

- Use dot (.) operator.

s1.id → 101

s1.name → "CoderStar"

Why Structures?

- ☑ Better organization
- ☑ Easy to handle
- ☑ Real-world modeling

Real Life Analogy

Student ID Card

ID : 101

Name : CoderStar

Structure is like a blueprint to create many student ID cards!

2 FILE HANDLING

→ File handling allows our program to read from or write to files.

data.txt

Basic Steps

1. Open File
2. Perform Operations
3. Close File

File Pointer created!

Example

```
FILE *fp;
fp = fopen("data.txt", "w"); // open file for writing
```

Common Functions

Function	Use	Example
fopen()	Open a file	fp = fopen("data.txt", "r");
fclose()	Close a file	fclose(fp);
fprintf()	Write formatted data to file	fprintf(fp, "ID: %d", 101);
fscanf()	Read formatted data from file	fscanf(fp, "%d", &id);
fgets()	Read a line of text from file	fgets(str, 50, fp);
fputs()	Write a string to file	fputs("Hello\n", fp);

File Operations Flow

Start → Open File fopen() → Read / Write Data → Close File fclose() → End

File Modes

- r → Read
- w → Write
- a → Append
- r+ → Read + Write
- w+ → Write + Read

Benefits

- ☑ Structures → Organize diverse data efficiently.
- ☑ File Handling → Store and manage data permanently.

Good Code + Organized Data + File Power = Great Programs!

C Notes

Practice Today
Build Tomorrow
Succeed Always!